Future Generation Computer Systems 185 (2026) 108700



Contents lists available at ScienceDirect

Future Generation Computer Systems

journal homepage: www.elsevier.com/locate/fgcs





A reference architecture for an orchestrated collaborative MLOps: A ground-up design from literature combining human and LLM expertise☆

Charbel Daoud [a],*, Danielle Azar [b], Christelle Urtado [a], Sylvain Vauttier [a]

[a] SyCoIA, IMT Mines Ales, Ales, France
[b] Department of Computer Science & Mathematics, Lebanese American University, Byblos, Lebanon

ARTICLE INFO

Keywords:
Software engineering
Machine learning operations
MLOps actors
MLOps activities
MLOps Reference architecture
Collaborative MLOps
Orchestration

ABSTRACT

With the increasing adoption of Machine Learning (ML), the need to manage, in a tailored manner, the end-to-end engineering of ML-powered software arises. Machine Learning Operations (MLOps) extends DevOps to address the specific challenges of the development and lifelong management of this new type of software. Despite the growing attention to this domain, an accepted, accessible, and modular reference architecture for MLOps is still missing. This paper presents SkeltyMLOps, a reference architecture aimed at promoting collaboration among the diverse actors involved in MLOps. SkeltyMLOps is built as rigorously as possible from knowledge gathered through a systematic literature review. First, sets of MLOps actors and activities are manually extracted from the literature by two independent authors. The extracted terms are then clustered and normalized through a process involving human and Large Language Model (LLM) experts. To carefully build the artificial expertise, several experiments are conducted using various LLMs, with or without additional contextual knowledge and under different parameter settings. The resulting clusters and labels serve as the foundation for the design of the SkeltyMLOps architecture. There, special attention is paid to designing the orchestration of MLOps activities and to the coordination of contributions of various actors. Therefore, the originality of the contribution is twofold: (1) the careful knowledge gathering process that grounds the proposed architecture, and (2) the central role given to orchestration and coordination at the heart of the architecture.

1. Introduction

Machine learning (ML) is increasingly embedded in software [5–7], where the value of ML-powered software depends not only on model performance, but also on the ability to ship, operate, and evolve the underlying artifacts, including data, artificial intelligence (AI) models, and code, reliably over time [5,8,9]. Once deployed, AI models might face changing data distributions, shifting user behavior, and evolving operational constraints. Consequently, performance may degrade without explicit failures or produce unexpected behavior [10,11]. This makes monitoring, automation, reproducibility, and the reliable evolution of ML-powered software key engineering concerns [9]. In response, Machine Learning Operations (MLOps) has emerged by borrowing and adapting DevOps practices to address the end-to-end engineering process of ML-powered software, extending operationalization concerns to both AI models and data [12–14].

Despite the growing adoption of MLOps tools and platforms, practitioners still face a persistent engineering gap: MLOps solutions are often assembled as heterogeneous software whose integration and governance remain complex, costly, and difficult to adapt [1,15,16]. In such settings, integration decisions should be considered as coordination decisions: specialized actors should align on ownership boundaries and handoff protocols. As a result, breakdowns frequently arise at cross-actor boundaries, where responsibilities are ambiguous and dependencies across data, models, and software are not explicit or managed [12,13,17,18].

This paper[1] addresses this gap by proposing SkeltyMLOps, a software reference[2] architecture for an orchestrated collaborative MLOps. Before designing the architecture, a systematic literature review is conducted to better define MLOps in terms of its actors and activities. To consolidate the terms extracted from the literature review, human

☆ This article is part of a Special issue entitled: 'MLOps' published in Future Generation Computer Systems.

* Corresponding author.
E-mail addresses: charbel.daoud@mines-ales.fr (C. Daoud), danielle.azar@lau.edu.lb (D. Azar), christelle.urtado@mines-ales.fr (C. Urtado), sylvain.vauttier@mines-ales.fr (S. Vauttier).

[1] This paper is an extended version of a paper published at MLOps@ECAI [1].

[2] A reference architecture provides a reusable template of structures, elements, and relationships for building software within a given domain [2–4].

https://doi.org/10.1016/j.future.2026.108700
Received 31 March 2026; Received in revised form 1 July 2026; Accepted 2 July 2026
Available online 11 July 2026
0167-739X/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).



Fig. 1. The DevOps loop inspired by [19].

Table 1
Scope of existing secondary studies in the MLOps domain. ✓ = primary concern; ∘ = partial; – = not addressed.

Study	#	Type	Act.	Activ.	Coord.	Arch.
Kreuzberger et al. [13]	27	Mixed	✓	✓	∘	✓
Najafabadi et al. [28]	43	Syst. map.	–	–	–	✓
Eken et al. [12]	198	Multivocal	✓	✓	∘	–
Wozniak et al. [29]	41	Syst. rev.	–	✓	–	✓
This study	53	Syst. rev.	✓	✓	✓	✓

and LLM (Large Language Model) experts jointly cluster and normalize the terminology. Based on these results, the reference software architecture is designed from the ground up, with components aligned with the responsibilities of MLOps actors. Collaboration between actors and coordination across components are supported through an MLOps Process Orchestrator.

The contributions of this paper are:

- a structured characterization of MLOps from the literature, consolidating dispersed terminology into a clear set of actors, activities, and dimensions;
- a consolidation process involving human and LLM experts that supports the normalization and clustering of actors and activities extracted from the literature;
- SkeltyMLOps, a modular reference software architecture for orchestrated MLOps designed to support collaboration among the various actors involved and centered on tickets, handoffs, and coordination contracts;
- a comparative coverage analysis against existing MLOps architectures in the literature that assesses the extent to which the proposed architecture captures the identified MLOps activities and key collaboration concerns.

The remainder of this paper is structured as follows. Section 2 presents the problem framing, study design, and research questions. Section 3 describes the Phase 1 methodology. Section 4 reports the actors, dimensions, and activities extracted from the literature, and revisits the MLOps loop. Section 5 introduces SkeltyMLOps and evaluates it against representative architectures from the literature. Section 6 presents the reproducibility package, including the prompts, the evaluated LLM configurations, and the generated outputs. Finally, Section 7 concludes the paper.

2. Problem framing and study design

Development Operations (DevOps) bridges software development and operations to shorten software release cycles by formalizing automation and fast feedback loops [20,21]. It is commonly depicted as an iterative loop that links development activities (Plan, Code, Build, Test) with operational activities (Release, Deploy, Operate, Monitor), where the monitoring of the software in production informs subsequent requirements refinement, planning, and implementation [22,23] (see Fig. 1). At a high level, this loop spans two dimensions[3]: Dev and Ops.

In ML-powered software, this loop must account for additional sources of change compared to traditional software. After deployment, it is possible for the data distribution to change, features to become outdated, and model performance to degrade. Foundational work on ML technical debt argues that these dynamics create hidden maintenance costs and operational risks unless software supports monitoring, reproducibility, and disciplined change management [9,24]. Consequently, MLOps extends DevOps by redefining both development and operations around code, data, and models, together with mechanisms such as versioning of data and models, experiment tracking, model registry, continuous evaluation, and (when appropriate) retraining and redeployment.

Although literature and industry offer many tools and platform stacks, the architectural guidance for practitioners remains fragmented. Empirical and secondary studies report recurring challenges: the absence of unified architectural guidance, the complexity of integrating heterogeneous components, and the limited modularity of many commercial solutions [15,25,26]. Studies of open-source practices further indicate that accessible and reusable reference architectures remain scarce, complicating the structured adoption of MLOps [27].

Several secondary studies have surveyed the MLOps field (Table 1). Kreuzberger et al. [13] combine a literature review, a tool survey, and expert interviews to identify MLOps principles, components, and actors. Najafabadi et al. [28] map MLOps architectures and synthesize 35 architectural components through a systematic mapping study. Eken et al. [12] conduct a multivocal review covering practices, actors, activities, challenges, and open issues. Wozniak et al. [29] review MLOps components, tools, and process steps. Although some of these studies report actors or activities, none performs a systematic extraction and consolidation that normalizes the heterogeneous terminology found across the corpus into a unified characterization linking actors to their activities. In particular, coordination across actors, including ownership boundaries, handoff specifications, and governance mechanisms, is not treated as a primary concern in any of these studies. This gap motivates the present SLR: a literature analysis that extracts and consolidates actors and activities jointly, so that the resulting evidence can serve as the foundation for a reference architecture centered on coordination.

Reference architectures are valuable in this context because MLOps software must reconcile multiple concerns simultaneously: (i) technical concerns (data management, deployment, monitoring), (ii) organizational concerns (separation of responsibilities between specialized actors), and (iii) evolution concerns (controlled change across data, model, and software artifacts). Before designing a reference architecture for MLOps, it is first necessary to clarify what MLOps covers in the literature. Indeed, although the term is widely used, the field still lacks a universally adopted and clear definition. Existing studies describe MLOps through varying terminology, which creates ambiguity regarding its main actors and constitutive activities. This lack of clarity makes the design of a reference architecture difficult, since the architecture must rest on an explicit and well-structured understanding of the domain it aims to support.

For this reason, this work is organized into two phases, as illustrated in Fig. 2. The first phase aims to establish an understanding of MLOps through a systematic literature review. It identifies how MLOps is defined and characterized in the literature, with particular attention to the actors involved and the activities reported in MLOps processes. The second phase builds on these results to design a modular reference architecture that reflects the evidence gathered in the first phase. Accordingly, the study is guided by the following research questions.

[3] In this paper, a dimension denotes a distinct functional area that groups related actors and responsibilities.

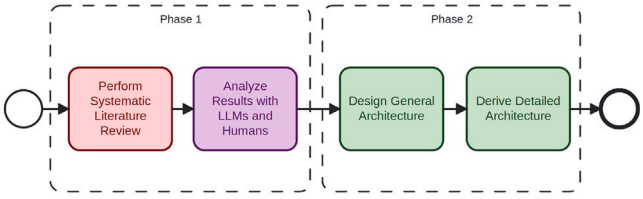


Fig. 2. Overview of the study design, structured in two phases.

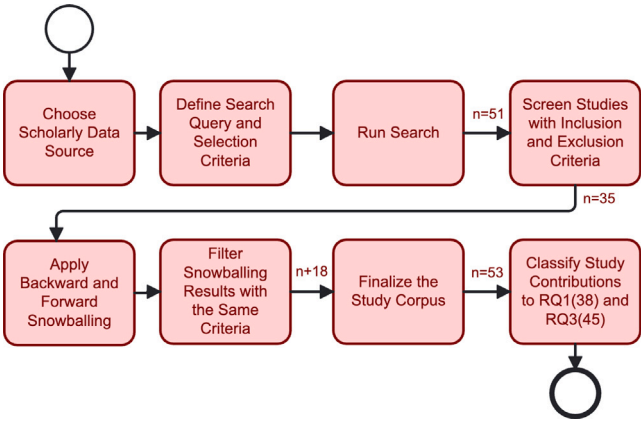


Fig. 3. Details of the `Perform Systematic Literature Review` activity.

Phase 1: Literature Analysis Review

- **RQ1:** Who are the main actors involved in MLOps?
- **RQ2:** What are the dimensions of MLOps?
- **RQ3:** What are the activities that constitute MLOps processes?

Phase 2: Orchestration and Collaboration

- **RQ4:** How can a modular MLOps reference architecture be designed to integrate the activities identified in RQ3 and to facilitate collaboration among the actors identified in RQ1?

3. Literature analysis review methodology

As discussed in the previous section, the lack of consistent definitions and taxonomies of MLOps in the literature motivates a two-phase study design. The first phase, presented in this section, focuses on the literature analysis methodology to clarify key MLOps concepts and to establish the conceptual basis for the ground-up design of the reference architecture carried out in Phase 2. Figs. 3 and 4 provide an overview of the methodological process followed in this first phase.

3.1. Systematic literature review (SLR) methodology

The SLR methodology follows Kitchenham's guidelines for systematic literature reviews in software engineering [30–32]. The search[4] is performed using Google Scholar to broaden coverage across publishers and venues, as recommended by previous comparative analyses [34].

[4] Conducted in March 2025. Raw data have been published and are available at [33].

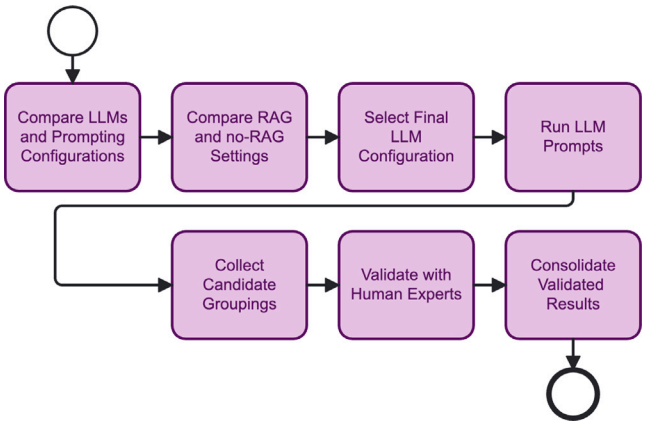


Fig. 4. Details of the `Analyze Results with LLMs and Humans` activity.

In particular, Google Scholar functions as a meta-indexer that scrapes publisher sites for metadata, so that a single query covers content from IEEE, ACM, Springer, Elsevier, and other major publishers without requiring separate searches adapted to the query language of each database [35]. The publisher distribution of the resulting corpus, IEEE (36%), Springer (9%), Elsevier (8%), MDPI (8%), and ACM (6%), among others, confirms that this coverage is effective across venues. All retained PDFs are retrieved manually from the publisher sites, and each paper is verified to be a peer-reviewed publication. To support reproducibility, the resulting papers are exported and screened using predefined inclusion and exclusion criteria, reported below.

The search query is constructed from terms derived from the first three research questions (MLOps, actors, activities, processes, architecture) and is iteratively refined with common synonyms. The final search string is:

```
allintitle:(MLOps OR "machine learning operations"
) AND (actor OR actors OR pipeline OR pipelines OR
architecture OR architectures OR architecting OR
activity OR activities OR workflow OR workflows OR
process OR processes)
```

The use of `allintitle` emphasizes precision by targeting papers that explicitly position themselves in the MLOps space. Its limitations are addressed through complementary screening and citation-based expansion in the subsequent steps.

3.2. Research paper selection

To calibrate the screening decisions, two authors independently screen the same set of 10 pilot papers and resolve disagreements in a consensus meeting to finalize the interpretation of the inclusion and exclusion criteria and the associated coding rules [32,36]. A paper is included if it explicitly reports at least one extractable artifact: (i) a stated set of MLOps actors, (ii) a process depiction of MLOps (*e.g.*, a process figure), or (iii) an explicit enumeration of MLOps-related activities. This criterion is evaluated at the content level: actors, processes, or activities are extracted from text, figures, or tables. Papers are excluded if they are (a) non-English, (b) records without accessible full text (*e.g.*, citation-only), (c) non-peer-reviewed at the time of the study, or (d) available only via subscription sources not accessible during the review period.

The initial search yields 51 papers. After applying the inclusion and exclusion criteria, the dataset contains 35 primary papers. To mitigate the limitations of a title-based query, backward and forward

Table 2
Manually extracted MLOps actors from the selected studies.

Actor	Citations	Count
Business Stakeholder	[13,37,38]	3
Solution Architect	[13,38]	2
Domain Expert	[37,39–42]	5
Manager	[37,40]	2
Data Scientist	[13,29,37,39,40,43–54] [38,41,55–66]	31
Data Engineer	[13,37–41,43,44,46,48,49,52,55,57,62, 65,67]	17
Data Steward	[41,52]	2
ML Engineer/Developer	[43,45,49–51,61,63,68,69]	9
Scientist	[37,70]	2
Software Developer	[58,67–69]	4
Software Engineer	[13,37–39,41,59,61,67]	8
Application Developer	[40,42,54]	3
DevOps Engineer	[13,37,38,41,43,45,61,62]	8
MLOps Engineer	[13,37,38,41,52,59]	6
Operations Engineer	[39,41,46,48,50,56,61]	7
IT Operations/Professional	[45,62,67]	3

snowballing[5] [71–73] is then applied. All additional papers are filtered using the same criteria, yielding 18 additional papers and resulting in a final set of 53 selected papers. Not all retained papers contribute equally to each research question: among the 53 selected papers, 38 contribute to RQ1 and 45 contribute to RQ3, with 30 papers addressing both.

3.3. Extraction protocol and discussion

For each retained paper, the following are recorded: (i) bibliographic metadata, (ii) explicitly mentioned actors (RQ1), and (iii) activities (RQ3) extracted from text or process figures. The extraction follows a content analysis approach guided by predefined categories [74] and is performed manually and independently by two authors, and disagreements are resolved by consensus [32,36]. Pre-consensus inter-rater agreement is measured using Cohen's kappa on a binary (present/absent) matrix: after synonym normalization, all unique items across all papers form the reference list (38 actor names and 58 activity types), and each rater's extraction is coded per (paper, item) pair. Cohen's kappa is $\kappa = 0.808$ for actors and $\kappa = 0.780$ for activities, both indicating substantial agreement according to the Landis and Koch scale [75], computed before the consensus meeting. The traceability between the extracted artifacts and their sources is maintained in a dataset [33].

Before any consolidation, this extraction step yields 16 actor names and 402 activities. These are summarized as a checkpoint for assessing terminology heterogeneity, quantifying the volume of extracted material, and clarifying the normalization effort required before deriving stable actors, dimensions, and activities. Table 2 reports the full list of extracted actor names, while the complete set of extracted activities is provided in the dataset due to space limitations [33].

[5] Backward and forward snowballing identifies relevant papers through cited and citing papers.

In the following, the extracted artifacts are examined to assess heterogeneity of terminology and responsibility granularity across the corpus.

Actors

Variability in actor names. Actor names exhibit substantial variability in naming and granularity across papers, including both near-synonyms and overloaded names.

Model-centric vs. software-centric actors. Many papers [13,59,61] distinguish actors primarily associated with model development (*e.g.*, *Data Scientist*, *ML Engineer*) from actors primarily associated with software delivery and operations (*e.g.*, *Software Engineer*, *DevOps Engineer*), indicating a boundary between experimental model work and production software responsibilities.

Cross-cutting actors. Few papers [42,70] describe actors whose responsibilities span multiple phases rather than a single step (*e.g.*, *Domain Expert* contributing to requirements and acceptance criteria, data labeling or validation, and error analysis), often coordinating with specialized technical actors.

Limited visibility of business actors. Business actors (*e.g.*, *Business Stakeholder*, *Product Owner*) are explicitly mentioned in only 7 of the 38 selected papers that contribute to RQ1 [13,37–42], despite DevOps emphasizing planning and prioritization activities [19,23].

Inconsistent naming. Different names refer to similar responsibility sets, and the same name can denote different responsibilities between papers. For example, a model-building actor is named *Data Scientist* in some papers [13,37,39,43,44,46,48,52,55,57,65] and *ML Engineer/Developer* in others [43,45,51,61,63,68,69], with overlapping responsibilities (*e.g.*, feature engineering, training, evaluation). Similarly, *Software Engineer* may refer to an actor focused on software components and model serving [13,38,39,41,59,61] or to a generalist covering most activities end-to-end [69].

Variable responsibility granularity. Papers differ in how they partition responsibilities: some assign broad, multi-area work to a single actor (*e.g.*, one actor covering both data preparation and model development [43,44,52]), whereas others distribute similar work across specialized actors.

Activities

Volume and heterogeneity. The extraction yields 402 activity statements across the selected papers. These activities vary in specificity

and phrasing, ranging from fine-grained tasks (*e.g.*, concrete implementation steps) to coarse-grained process stages, which makes manual consolidation into a stable activity list time-consuming and difficult to reproduce.

Altogether, the actor ambiguity and the activity volume motivate an explicit consolidation procedure to normalize names and propose candidate abstractions before deriving dimensions and finalized actor and activity lists. The next subsection therefore introduces human and LLM experts working jointly to analyze candidate groupings and propose merges under author verification.

3.4. Human and LLM experts analysis process

LLMs are not used in the extraction step; the extraction of actors and activities from each paper is performed manually by two authors (*cf.* Section 3.3). LLMs are introduced only in the subsequent consolidation step. The extracted artifacts indicate variability in terminology and responsibility granularity across papers, making manual cross-paper consolidation time-consuming and difficult to reproduce at corpus scale. Therefore, human and LLM experts are brought together in the analysis process to propose candidate semantic groupings and intermediate abstractions for consolidation, with LLM experts acting as analysis aids rather than autonomous decision makers [76,77]. Throughout this analysis process, all outputs are treated as provisional suggestions and undergo human expert verification through negotiated agreement [78] before inclusion in the final synthesis. This assisted synthesis transforms the extracted artifacts into a structured representation of MLOps to (i) identify recurring actors (RQ1), (ii) derive higher-level dimensions (RQ2), and (iii) consolidate extracted activities (RQ3).

LLM experts are used mainly for two purposes: (i) clustering the extracted artifacts that express similar semantics despite lexical variation, and (ii) proposing names for the resulting clusters and artifact categories. The same structure is used for all prompts: each prompt specifies the objective, the artifact under analysis, the constraints, and the output schema [79,80]. Then, human experts revise or reject inadequate LLM outputs (unsupported groupings, misleading names, or overly general categories) and document the resulting changes, ensuring that the final list remains internally consistent. A Human Modification Log documenting every case where the authors overrode the LLM proposal, including the modification type, the proposed item, the final item, and the justification, is available in the replication package. The use of LLMs in this step is motivated by the volume and heterogeneity of the extracted artifacts: 402 activity statements with diverse phrasing make manual consolidation time-consuming and difficult to reproduce, as different researchers may reach different groupings. LLMs provide a reproducible consolidation baseline that human experts then validate and refine. The detailed five-step consolidation procedure, including the exact prompts used, is described in Section 4.3.

3.5. LLM comparison design

LLMs and their configurations must be compared explicitly to identify a configuration suited to the study's analysis tasks.

The comparison considers LLM configurations from four providers: OpenAI, Anthropic, Google, and DeepSeek. Table 3 reports the evaluated LLMs together with their identifiers, interaction mode, and temperature settings.

Temperature is a parameter that controls output variability: lower values produce more deterministic and stable outputs, whereas higher values increase variability and diversity. Some LLMs, such as `o3`, `o4-mini`, and `deepseek-reasoner`, are only available with their default temperature. For the others, the main temperatures used are 0 and 0.7. In two cases (Claude Sonnet 4.6, Gemini 2.5 Flash), a temperature of 1 is also reported because some providers use it to activate a higher-effort reasoning mode. A reasoning mode gives the LLM more computation before answering, which may improve results on harder tasks but usually increases latency.

Candidate LLM configurations are compared using a single task and the same metrics. In this task, each LLM is asked to (i) propose dimensions, (ii) assign actors to these dimensions, and (iii) identify a leading actor for each dimension. Readers are referred to Section 4.3 for the exact prompt used in this task. Each configuration is run 10 times.

The comparison examines completeness, validity, structural consistency, and stability while also checking whether the outputs follow the required schema. The metrics used are as follows:

Coverage (%) measures how many actors from the extracted actor list appear in an LLM output.

Invalid items per run reports the mean number of items outside the extracted actor list.

Duplicate assignments per run measures the mean number of cases in which the same actor is assigned to more than one dimension within a single run.

Mean number of dimensions and **standard deviation of the number of dimensions** describe, respectively, the average number of dimensions and how much this number changes over repeated runs.

3.6. Results and selection of the champion LLM

Table 4 summarizes the metric values obtained for each configuration for the fixed prompt. Because all configurations achieve full coverage (100%) except `o4-mini` (98.75%), the selection is driven primarily by correctness and stability measured through (i) zero duplicate assignments, (ii) low variability in the number of dimensions across runs, and (iii) consistent schema output.

Among the configurations achieving full coverage, `gemini 2.5 flash` at temperature 0 achieves the most consistent structure, with zero invalid items, zero duplicate assignments, and a stable dimension count (mean 6, standard deviation 0.00). Other configurations either introduce duplicate assignments (*e.g.*, `gpt 4o`, `o3`, and `claude sonnet` at temperatures 0 and 0.7), or exhibit higher variation between runs in the number of dimensions (*e.g.*, `deepseek reasoner`, `gpt 4o`, `o3`). Although `deepseek chat` at temperature 0 also yields zero duplicates, it produces a coarser clustering (mean 3 dimensions) and shows non-zero variability (0.32), which is less suitable for the target granularity used to structure subsequent activity mapping and architectural decomposition. Higher-temperature settings for `gemini 2.5 flash` produce more instability and more duplicate assignments (2–4 duplicates; standard deviation 1.17–1.25), which further supports using temperature 0 for consolidation.

Based on these criteria, `gemini 2.5 flash` at temperature 0 is selected as the champion LLM for the next synthesis steps. The next subsection then examines whether adding Retrieval-Augmented Generation (RAG) further improves the consolidation quality under the same prompt and model setting. RAG gives the model access to the retrieved external context, such as source documents, so that it can base its output on explicit context instead of only its internal knowledge.

3.7. RAG vs. no-RAG settings

For the selected champion LLM, Gemini 2.5 Flash at temperature 0, retrieval support is assessed by comparing a RAG configuration with a no-RAG configuration under identical prompt and settings. The RAG setting provides the model with the extracted artifacts and full PDF papers from the corpus, whereas the no-RAG setting provides only the prompt and the extracted artifacts. This comparison isolates the effect of external grounding on consolidation quality.

Both configurations achieve 100% actor coverage, indicating that retrieval does not improve the primary completeness criterion. However, the RAG configuration consistently introduces an additional actor,

Table 3
The thirteen LLM configurations compared in the synthesis study.

Provider	Model	Mode	Identifier	Temperature	Website
Anthropic	Claude Sonnet 4.6	Chat	`claude-sonnet-4-6`	0	www.anthropic.com
Anthropic	Claude Sonnet 4.6	Chat	`claude-sonnet-4-6`	0.7	www.anthropic.com
Anthropic	Claude Sonnet 4.6	Reasoning	`claude-sonnet-4-6`	1	www.anthropic.com
DeepSeek	DeepSeek Chat	Chat	`deepseek-chat`	0	www.deepseek.com
DeepSeek	DeepSeek Chat	Chat	`deepseek-chat`	0.7	www.deepseek.com
DeepSeek	DeepSeek Reasoner	Reasoning	`deepseek-reasoner`	Default	www.deepseek.com
Google	Gemini 2.5 Flash	Chat	`gemini-2.5-flash`	0	www.ai.google.dev
Google	Gemini 2.5 Flash	Chat	`gemini-2.5-flash`	0.7	www.ai.google.dev
Google	Gemini 2.5 Flash	Reasoning	`gemini-2.5-flash`	1	www.ai.google.dev
OpenAI	GPT-4o	Chat	`gpt-4o`	0	www.openai.com
OpenAI	GPT-4o	Chat	`gpt-4o`	0.7	www.openai.com
OpenAI	o3	Reasoning	`o3`	Default	www.openai.com
OpenAI	o4-mini	Reasoning	`o4-mini`	Default	www.openai.com

Table 4
Comparison of evaluated model configurations.

Identifier	Runs	RAG?	Temp.	Coverage (%)	Invalid Items	Duplicates	Mean # Dim.	Stdv. Dim.
`claude-sonnet-4-6`	10	×	0	100	0	1	6	0.82
`claude-sonnet-4-6`	10	×	0.7	100	0	1	6	0.79
`claude-sonnet-4-6`	10	×	1	100	0	0	7	2.96
`deepseek-chat`	10	×	0	100	0	0	3	0.32
`deepseek-chat`	10	×	0.7	100	0	1	3	0.00
`deepseek-reasoner`	10	×	Default	100	0	0	5	0.57
`gemini-2.5-flash`	10	×	0	100	0	0	6	0.00
`gemini-2.5-flash`	10	×	0.7	100	0	4	6	1.17
`gemini-2.5-flash`	10	×	1	100	0	2	6	1.25
`gemini-2.5-flash`	10	✓	0	100	1	0	6	0.00
`gpt-4o`	10	×	0	100	0	1	5	1.29
`gpt-4o`	10	×	0.7	100	0	1	5	1.07
`o3`	10	×	Default	100	0	1	7	0.74
`o4-mini`	10	×	Default	98.75	0	0	6	0.71

Table 5
Derived MLOps dimensions and actor clusters.

Proposed dimensions	Actor clusters	Leading actor
Plan	Business Stakeholder, Solution Architect, Manager, Domain Expert	Product Owner
Data Engineering (Data)	Data Scientist, Data Engineer, Data Steward	Data Engineer
Model Engineering (Model)	ML Engineer/Developer, Scientist	Model Engineer
Software Engineering (Software)	Software Engineer, Software Developer, Application Developer	Software Engineer
Operations (Ops)	DevOps Engineer, MLOps Engineer, Operations Engineer, IT Operations/Professional	Operations Engineer

`ML Architect`, which is not part of the predefined extracted actor list and is therefore counted as invalid (see invalid items column). This actor name is not retained in the final actor list (provided in the prompt) because it does not emerge as a sufficiently recurrent actor in the reviewed corpus and is removed during expert validation. By contrast, the no-RAG configuration preserves full coverage while producing zero invalid items and zero duplicate assignments. Since the task is defined as consolidating actors against a fixed reference list, strict adherence to that schema is prioritized in the final LLM selection. As a consequence, the no-RAG configuration is retained as the final configuration.

3.8. Threats to validity

Despite the structured protocol adopted in this phase, several threats may affect the interpretation, robustness, and transferability of the results.

Internal validity. The consolidation of actor names, dimensions, and activities relies in part on analysis by LLM experts, which may introduce unsupported merges, unstable names, or structurally inconsistent output. This threat is mitigated through independent manual extraction by two authors with consensus resolution, structured prompts, repeated runs, comparison of multiple providers and settings, and systematic human verification of all LLM outputs. Retrieval support is also explicitly assessed through a controlled RAG versus no-RAG comparison. Still, the final consolidated lists remain partially influenced by human expert judgment, especially when handling infrequent or ambiguous names.

External validity. The corpus is restricted to peer-reviewed papers retrieved primarily through Google Scholar using an `allintitle:` query, and then expanded through backward and forward snowballing. Although this improves coverage/completeness, relevant papers may still be missed because of terminology choices, indexing limitations, or full-text access constraints. In addition, excluding non-peer-reviewed sources means that some industrial practices are not represented. The resulting actor and activity lists should therefore be interpreted as literature-grounded rather than exhaustive.

Reliability and reproducibility. Parts of the analysis depend on commercial LLM services, whose outputs may vary between providers, LLM versions, and settings. To mitigate this threat, each configuration is executed repeatedly over time, variability is reported explicitly, and prompts, settings, outputs, and implementation artifacts are maintained in a replication package. However, exact long-term reproducibility may

be limited by future changes in LLM endpoints, provider behavior, or search-engine results.

Conclusion validity. The conclusions rely on descriptive evidence, including mention counts, frequency of extracted names, and comparison metrics computed on a fixed task, all documented in the replication package. These indicators support comparative analysis, but do not imply that the resulting consolidation is the only valid interpretation of the corpus. The claims are therefore restricted to the selected corpus, the extraction protocol, and the reported comparison criteria.

4. Literature-derived view of mlops

This section reports the results of Phase 1. It synthesizes the selected literature into consolidated MLOps actors, dimensions, and activities through the combined analysis of human and LLM experts, following the extraction and consolidation procedure described in the previous section.

4.1. Actors and dimensions (RQ1 & RQ2)

This subsection reports (i) the consolidated actor list extracted from the corpus and (ii) the dimension scheme derived by clustering these actors. Dimensions serve two roles: (i) they structure the reporting of activities (RQ3), and (ii) they inform the modular decomposition of SkeltyMLOps (RQ4) by aligning architectural packages with responsibility boundaries.

Dimensions are derived by clustering the consolidated actor list using the LLM-assisted analysis process described in Section 3. In this task, the model receives the complete list of actors extracted from the literature (*cf.* Table 2) and is instructed to (i) propose a list of dimensions, (ii) assign actors to dimensions, and (iii) identify a leading actor for each dimension. The prompt used for this task is shown below.

```
You're an expert in Machine Learning Operations (MLOps). You will find below a list that contains 16 Actors who appear in different activities in an MLOps process. I want you to:
(1) Identify dimensions from the provided actors.
(2) Cluster actors according to these dimensions.
(3) Propose a leading actor name for each dimension.
(4) Format the output as a table with three columns: Dimension Name, Actor Cluster, Leading Actor.
```

Table 5 reports the resulting dimensions, which span the end-to-end engineering of ML-powered software. The proposed dimensions and assignments are then reviewed by human experts, who adjust the naming where needed and resolve inconsistencies or contradictions in the clusters. Table 5 presents the final validated structure.

4.2. Activities (RQ3)

This subsection addresses RQ3 by deriving a consolidated list of MLOps activities from the reviewed papers. Building on the dimensions introduced above, the analysis organizes the extracted activities into the same responsibility structure to obtain a coherent view of actors, dimensions, and activities. The objective is not only to enumerate reported activities, but also to normalize heterogeneous formulations, reduce redundancy, and produce a stable activity list grounded in the literature.

Because the selected studies do not provide a shared and standardized activity list, the extracted activity lists require an additional consolidation process. This process proceeds in five steps: (i) assignment of extracted activities to the validated dimensions, (ii) dimension-specific clustering of semantically related activities, (iii) redundancy reduction and label normalization, (iv) expert review and final merging, and (v) enrichment of the resulting list with descriptions and frequency metadata.

4.3. Deriving a consolidated activity list

(1) Assignment to validated dimensions. Following the derivation of the validated dimensions and their leading actors in Table 5, each extracted activity is assigned to one of the same dimensions to preserve coherence between the responsibilities of the actors and the structuring of the activities. To reduce manual effort, limit labeling inconsistency, and support reproducible classification at corpus scale, the champion configuration, `gemini 2.5 flash` at temperature 0, is used to classify the extracted activities into the validated dimensions. The prompt used for this classification is shown below.

```
You're an expert in Machine Learning Operations (MLOps). You will find, at the end, a list that contains extracted Activities which appear in different MLOps processes. I want you to:
(1) Categorize these Activities into the provided dimensions.
(2) Ensure activities are not mistakenly categorized.
(3) Format the final output as a structured table with two columns: Dimension, Activity.
```

The classification output remains provisional and is reviewed by human experts to correct misplaced assignments and ensure consistency with the responsibility boundaries established in the actor-and-dimensions analysis.

(2) Dimension-specific clustering. After dimension assignment and manual review, the activity list within each dimension is clustered separately. This preserves semantic coherence within each responsibility area and avoids premature merging of activities that may appear similar lexically but belong to different operational contexts.

(3) Redundancy reduction and normalization. For each dimension, a dedicated clustering and labeling prompt is applied to (i) group semantically similar activities, (ii) avoid merging conceptually distinct tasks, and (iii) produce concise and stable labels for the resulting clusters. This step reduces the initial list of extracted activities to a list of normalized candidate activities.

(4) Expert review and final merging. The normalized activity list is then reviewed by human experts to remove residual overlap, refine labels, and ensure that each activity meets the intended responsibility boundaries of its dimension. Minor refinements and merge decisions are documented in the replication package for traceability.

(5) Final consolidation with metadata. The final list associates each activity with (i) a short description, (ii) an indication of whether the activity also appears in DevOps processes, and (iii) its frequency of occurrence across the analyzed papers. These artifacts are reported in Table 6. This consolidation yields a final list of 38 activities.

4.4. List of 38 activities

Table 6 reports the resulting 38 consolidated activities, along with their brief descriptions (generated by the selected LLM), their overlap with DevOps (assessed by human experts), and their frequency, defined as the number of reviewed papers in which each activity appears based on the earlier extraction step.

The frequency distribution is concentrated. Twelve of the 38 activities appear in at most three papers, whereas four activities appear in at least 25 papers: *Train Model* (29), *Preprocess Data* (28), *Deploy to Production* (27), and *Evaluate & Test Model* (25). This concentration indicates that the literature repeatedly reports a relatively small activity core focused on data preparation, model training and evaluation,

Table 6
Proposed activities, brief descriptions, whether they also appear in DevOps, and number of occurrences in reviewed papers.

ID	Activity name	Brief description	Also in DevOps?	Count
P1	**Conceptualize Project**	Define business goal, context, and scope.	✓	5
P2	**Define Requirements**	Elicit requirements and constraints.	✓	2
P3	**Formulate ML Problem**	Translate the business goal into an ML problem.	×	3
P4	**Analyze Data Sources**	Identify required data and assess available sources.	×	3
P5	**Design Architecture**	Define system design, components, and technologies.	✓	3
P6	**Plan Project**	Plan work, milestones, and delivery approach.	✓	5
P7	**Initialize Repositories**	Set up version control and artifact repositories.	✓	3
D1	**Collect Data**	Acquire raw data from internal or external sources.	×	14
D2	**Ingest Data**	Load data into storage/pipeline (batch or streaming).	×	7
D3	**Validate Data**	Verify schema, integrity, and basic data quality.	×	7
D4	**Preprocess Data**	Clean and transform data for modeling.	×	28
D5	**Engineer Features**	Create and transform features for training/inference.	×	8
D6	**Label Data**	Create or refine labels/annotations.	×	2
D7	**Analyze Data**	Perform exploratory data analysis to assess issues.	×	10
D8	**Manage Data**	Version, catalog, and govern datasets and features.	×	3
M1	**Run Experiments**	Execute and track experimental runs and outcomes.	×	5
M2	**Design Model**	Select the model approach and key design choices.	×	10
M3	**Develop Model**	Implement the model in the chosen framework.	×	11
M4	**Train Model**	Fit the model on data; retrain as needed.	×	29
M5	**Tune Model**	Optimize hyperparameters and training settings.	×	3
M6	**Validate Model**	Check correctness and compliance against criteria.	×	11
M7	**Evaluate & Test Model**	Assess performance on test/holdout and analyses.	×	25
M8	**Optimize Model**	Improve efficiency/quality (*e.g.*, compression).	×	3
M9	**Register Model**	Store model artifact and metadata in a registry.	×	13
S1	**Develop Code**	Implement ML and non-ML software components.	✓	8
S2	**Build Components**	Build/containerize components and produce artifacts.	✓	12
S3	**Package Artifacts**	Bundle components/models into deployable artifacts.	✓	10
S4	**Test Solution**	Run checks and tests before release.	✓	12
S5	**Release Solution**	Publish artifacts to target repositories/environments.	✓	4
O1	**Configure CI/CD**	Set up automated build, test, and release pipelines.	✓	9
O2	**Manage Infrastructure**	Provision and configure runtime infrastructure.	✓	3
O3	**Execute Pipelines**	Run orchestrated workflows on schedule/trigger.	✓	7
O4	**Deploy to Testing**	Deploy to staging for validation.	✓	3
O5	**Deploy to Production**	Deploy validated release to production.	✓	27
O6	**Serve Inference**	Provide online/batch inference and log outputs.	×	11
O7	**Monitor Performance**	Track metrics, drift, and alerts in production.	✓	19
O8	**Trigger Retraining**	Start retraining based on signals or new data.	×	3
O9	**Operate & Maintain**	Handle incidents, feedback, and service upkeep.	✓	6

and production deployment, while the remaining activities appear less frequently.

A second observation concerns the overlap between DevOps activities and MLOps. In total, 17 of the 38 activities are marked as also appearing in DevOps, including repository setup, build, test, release, CI/CD configuration, deployment, monitoring, and operations. Among these, *Deploy to Production* (27) and *Monitor Performance* (19) are the most frequently reported, followed by *Build Components* (12), *Test Solution* (12), and *Package Artifacts* (10).

At the same time, several of the most frequently reported activities fall outside traditional DevOps, such as *Preprocess Data* (28), *Train Model* (29), *Evaluate & Test Model* (25), *Collect Data* (14), *Register Model* (13), and *Serve Inference* (11).

> Taken together, these results support the view that MLOps builds on a software-delivery backbone inherited from DevOps while adding explicit activities for evolving data and model artifacts.

The least-mentioned activities remain analytically important because they reveal concerns that are either under-reported in published process descriptions or left implicit. Using the first quartile as an operational definition of rarity (mentions in ≤ 3 papers), the rare activities include planning work (*Define Requirements* (2), *Analyze Data Sources* (3), *Label Data* (2), *Manage Data* (3)), infrastructure and pre-production steps (*Manage Infrastructure* (3), *Deploy to Testing* (3)), model refinement (*Tune Model* (3), *Optimize Model* (3)), and closed-loop operation (*Trigger Retraining* (3)).

> Their limited reporting contrasts with the higher frequency of training, evaluation, deployment, and monitoring, and indicates that several important responsibilities remain less visible in the literature despite their role in end-to-end evolution of ML-powered software.

Taken together, the actor list, the validated dimensions, and the consolidated activity list provide a literature-grounded representation of MLOps responsibilities and work structure. The next subsection summarizes this representation in process form before the discussion turns to architecture design.

4.5. The revisited MLOps loop

The consolidated actors, dimensions, and activities together support a synthesized view of the MLOps process. For process visualization purposes, the validated dimensions are summarized into the five-part loop shown in Fig. 5, while preserving their responsibility boundaries. Fig. 5 revisits the familiar DevOps-style loop and adapts it to MLOps by retaining the *Plan* and *Ops* dimensions while refining the engineering segment into three distinct dimensions: *Data*, *Model*, and *Software*. In this way, the loop makes explicit the additional responsibility structure introduced by data-intensive and model-centric work.

The figure groups the consolidated activities by dimension and annotates each dimension with its leading actor, as identified in Section 3. It therefore acts as a synthetic representation of the results reported in this section rather than as an additional derivation step. It also establishes a consistent color scheme for the dimensions, which is reused in subsequent figures to preserve visual continuity across the paper.

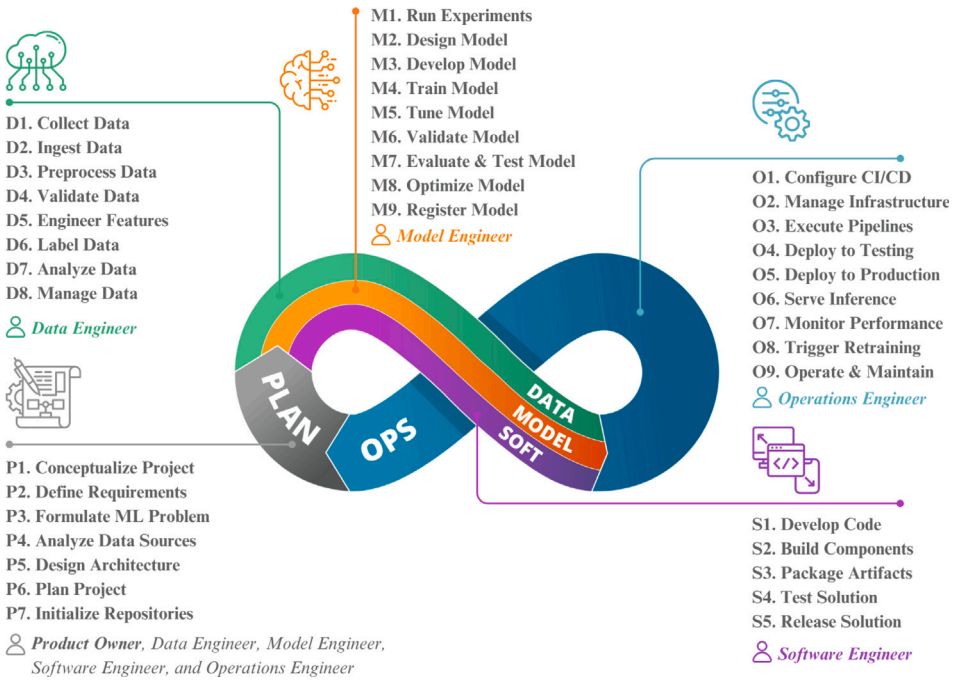


Fig. 5. The revisited MLOps loop: a synthesis of consolidated dimensions, leading actors, and activities.

With a clearer definition of MLOps and a consolidated view of its actors, dimensions, and activities, the next section proceeds to Phase 2 and derives the reference architecture.

5. The proposed SkeltyMLOps reference architecture

Phase 1 establishes a characterization of MLOps in terms of actors, dimensions, and activities. Based on these results, Phase 2 derives SkeltyMLOps as a reference architecture grounded in the dimension structure and the activity space identified in the literature. The objective of this phase is therefore not to propose an ad hoc architecture, but to design a reusable architectural blueprint from the bottom up, using the consolidated findings of Phase 1 as its foundation.

5.1. Related work

Before introducing the architecture itself, this phase is positioned with respect to representative MLOps architectures. Some studies propose architecture blueprints or reference models for particular settings, such as stream learning software, open source MLOps stacks, or industrial deployment pipelines [15,81,82]. Others synthesize the field through secondary studies, identifying recurring components, stages, and tool categories throughout the MLOps process [13,28]. In general, these studies clarify important aspects of MLOps practice, but vary substantially in abstraction level, scope, and intended contribution.

More specifically, previous work spans several complementary perspectives: specialized operational architectures, tool-oriented blueprints, industry-driven reference architectures, and survey-based overviews. These contributions help identify what MLOps software generally contains and which stages are commonly distinguished. However, they provide more limited guidance on how the various actors involved in MLOps coordinate work across ownership boundaries, approvals, handoffs, and runtime feedback loops. This limitation becomes particularly important when existing architectures are examined against the scope derived from the literature established in Phase 1. Concretely, the architectures are compared next to characterize which parts of the scope derived from the literature are made explicit by existing architectures and which remain implicit.

5.2. Comparison methodology

Five baselines are selected to represent distinct contributions in the MLOps architecture space: (i) an architecture for learning operations in near real-time [81], (ii) an open-source architecture blueprint [15], (iii) an industry-driven reference architecture with requirements and pipeline patterns [82], (iv) an architecture synthesized from a mapping study [28], and (v) a systematic mapping that consolidates MLOps components and process stages [13]. Because these baselines differ in the intended scope, the assessment is not interpreted as a quality ranking but rather as a structured comparison of explicitness with respect to a fixed scope derived from the literature.

Each architecture is scored against 44 elements: the 38 activities reported in the proposed consolidated list (Table 6) and six cross-cutting capabilities, namely *(i) explicit actor-to-responsibility mapping*, *(ii) cross-actor handoff specification*, *(iii) traceability across data, model, code, and operational artifacts*, *(iv) governance gates*, *(v) feedback loops (monitor → decision → action)*, and *(vi) software versioning*. The resulting matrix is reported in Table 7. For each baseline and element, a discrete score in $\{0, 1, 2\}$ is assigned using the following rubric:

Absent: the element is neither described nor supported by an architectural element or process activity, and cannot be reasonably grounded in the text or figures without speculation.

Partial: the element is mentioned or implied, but the architecture does not specify a concrete mechanism, such as a named module, a process activity, or actor ownership.

Explicit: the element is supported by an identifiable architectural element and/or a clearly described process activity with stated responsibilities and exchanged artifacts.

To support traceability, a rationale is recorded in a scoring sheet for every partial or explicit score, including the reference to supporting section or figure and a short justification grounded in the rubric. Two authors independently score all baselines using the same procedure: mapping components to elements from the architecture figure(s), confirming scope and responsibilities in the descriptive sections, and performing targeted keyword searches for high-signal terms such as *registry*, *versioning*, *CI/CD*, *monitoring*, *drift*, *pipeline*, *feature store*, and

Table 7
Coverage scores (–=absent, ∘ =partial, • =explicit) for activities across architectures.

ID	Activity	[81]	[15]	[82]	[28]	[13]
P1	Conceptualize Project	–	–	∘	–	•
P2	Define Requirements	–	–	–	–	•
P3	Formulate ML Problem	–	–	∘	–	•
P4	Analyze Data Sources	–	–	∘	–	•
P5	Design Architecture	•	•	•	•	•
P6	Plan Project	–	–	–	–	∘
P7	Initialize Repositories	–	∘	∘	∘	∘
D1	Collect Data	–	–	•	•	•
D2	Ingest Data	•	∘	•	•	•
D4	Preprocess Data	–	∘	•	•	•
D3	Validate Data	–	–	•	•	•
D5	Engineer Features	–	∘	•	•	•
D6	Label Data	•	–	∘	•	∘
D7	Analyze Data	–	–	–	–	•
D8	Manage Data	–	•	•	•	•
M1	Run Experiments	–	•	•	•	•
M2	Design Model	–	–	∘	–	•
M3	Develop Model	∘	–	∘	–	•
M4	Train Model	•	•	•	•	•
M5	Tune Model	–	∘	•	–	•
M6	Validate Model	–	•	•	∘	•
M7	Evaluate & Test Model	∘	∘	•	•	•
M8	Optimize Model	–	∘	∘	–	∘
M9	Register Model	•	•	•	•	•
S1	Develop Code	∘	∘	∘	∘	•
S2	Build Components	•	∘	∘	•	•
S3	Package Artifacts	•	•	•	•	•
S4	Test Solution	∘	∘	•	•	•
S5	Release Solution	–	∘	•	•	•
O1	Configure CI/CD	–	•	•	•	•
O2	Manage Infrastructure	•	•	•	∘	•
O4	Deploy to Testing	–	•	•	∘	∘
O5	Deploy to Production	∘	•	•	∘	•
O6	Serve Inference	•	•	•	•	•
O7	Monitor Performance	∘	•	•	•	•
O9	Operate & Maintain	∘	∘	∘	∘	•
O3	Execute Pipelines	•	•	•	∘	•
O8	Trigger Retraining	•	•	•	•	•
Cross-cutting capabilities						
–	Explicit Actor-to-Responsibility Mapping	–	∘	•	–	∘
–	Cross-actor Handoff Specification	•	•	•	•	•
–	Traceability Across Artifacts	∘	∘	•	∘	∘
–	Governance Gates	–	–	∘	–	∘
–	Feedback Loops	∘	•	•	•	•
–	Software versioning	–	∘	∘	–	∘

staging. Before consensus, the agreement is quantified using weighted Cohen's kappa with quadratic weights ($\kappa_w = 0.743$, substantial agreement [75]) and reported in the paper. Disagreements are then resolved through a consensus meeting using the recorded evidence notes; consensus affects only the final reported scores, not the pre-consensus agreement statistics.

5.3. Coverage metrics

To summarize the coverage matrix (*cf.* Table 7), three complementary indicators are reported: **total coverage**, computed by summing 0/1/2 scores respectively for absent, partial, and explicit across the 44 elements over the maximum possible score; **binary coverage**, computed as the proportion of elements receiving a score of 1 or 2 to reflect breadth independently of support strength; and **strong coverage**, computed as the proportion of elements scored 2 to reflect depth.

5.4. Comparative results and discussion

Table 8 reports conformance metrics for the five baselines with respect to the scope of 44 elements derived from the literature.

Table 8
Summary coverage metrics for representative MLOps architectures with respect to the scope derived from literature.

Architecture	Total coverage	Binary coverage	Strong coverage
[13]	89.8%	100%	80%
[82]	78.4%	93%	64%
[28]	60.2%	70%	50%
[15]	55.7%	73%	39%
[81]	37.5%	48%	27%

Kreuzberger et al. [13] attain full breadth (Binary coverage of 100%), consistent with the objective of providing an overview architecture; however, the corresponding Strong coverage (80%) indicates that multiple elements remain conceptual rather than operationalized through explicit modules or mechanisms. Rodrigues et al. [81] score the lowest in breadth and explicitness, consistent with a specialized architecture that targets near real-time stream learning operations rather than end-to-end MLOps. Burgueño-Romero et al. [15] and Najafabadi et al. [28] occupy the mid-range: both cover many core technical components, particularly in the Model and Ops dimensions, but remain comparatively weaker on planning activities and on cross-cutting coordination mechanisms such as governance gates, explicit ownership mapping, and software versioning. Kumara et al. [82] provide stronger end-to-end conformance among baselines, consistent with an industry-driven reference architecture; remaining gaps appear primarily in planning activities and in the complete specification of cross-cutting governance and software release management.

Beyond activity coverage, Table 8 also highlights the recurring differences between baselines in the six cross-cutting capabilities. Although several architectures support retraining, deployment, and monitoring, they often leave ownership boundaries, handoff payloads, approval logic, and release-level traceability implicit. In other words, much of the literature describes *what* technical components and stages are involved in MLOps, but offers less explicit guidance on *how* collaboration is organized and enforced throughout the process.

These recurring limitations directly motivate the architectural direction taken in SkeltyMLOps. Rather than treating coordination as an implicit by-product of tools or processes, SkeltyMLOps treats coordination as a key architectural concern. The reference architecture is therefore designed from the ground up to reflect the actor responsibilities, activities, and coordination needs derived in Phase 1.

5.5. Ground-up design rationale

This section introduces SkeltyMLOps as a reference architecture for MLOps explicitly grounded in the findings of Phase 1. As noted earlier, a reference architecture is a reusable blueprint that captures essential design principles, structural decomposition, and interface-level guidance for a domain, allowing multiple concrete instantiations [2–4]. SkeltyMLOps is organized along the five consolidated MLOps dimensions, which are refined into components that operationalize the associated activities.

5.6. Package-level view of skeltymlops

As shown in Fig. 6, the architecture is structured into five core packages (large colored rectangles), each corresponding to an MLOps dimension and encapsulating a coherent responsibility space. These packages are coordinated through a dedicated orchestration core. SkeltyMLOps serves as a reference architecture by grounding its decomposition in literature-derived dimensions and activities, while providing reusable guidance to integrate heterogeneous MLOps components through explicit responsibility boundaries and stable coordination mechanisms.

Table 9
Mapping between SkeltyMLOps components & activities.

Component name	Implemented activities
Project Management	Conceptualize Project, Define Requirements, Formulate ML Problem, Analyze Data Sources, Design Architecture, Plan Project, Initialize Repositories
Data Acquisition	Collect Data, Ingest Data
Data Processing & Feature Engineering	Preprocess Data, Validate Data, Engineer Features, Label Data, Analyze Data, Manage Data
Model Development & Training	Run Experiments, Design Model, Develop Model, Train Model, Tune Model, Validate Model, Evaluate & Test Model, Optimize Model, Register Model
API Development	Develop Code, Build Components, Package Artifacts, Test Solution, Release Solution
Deployment & Monitoring	Configure CI/CD, Manage Infrastructure, Deploy to Testing, Deploy to Production, Serve Inference, Monitor Performance, Operate & Maintain
MLOps Process Orchestrator	Execute Pipelines, Trigger Retraining

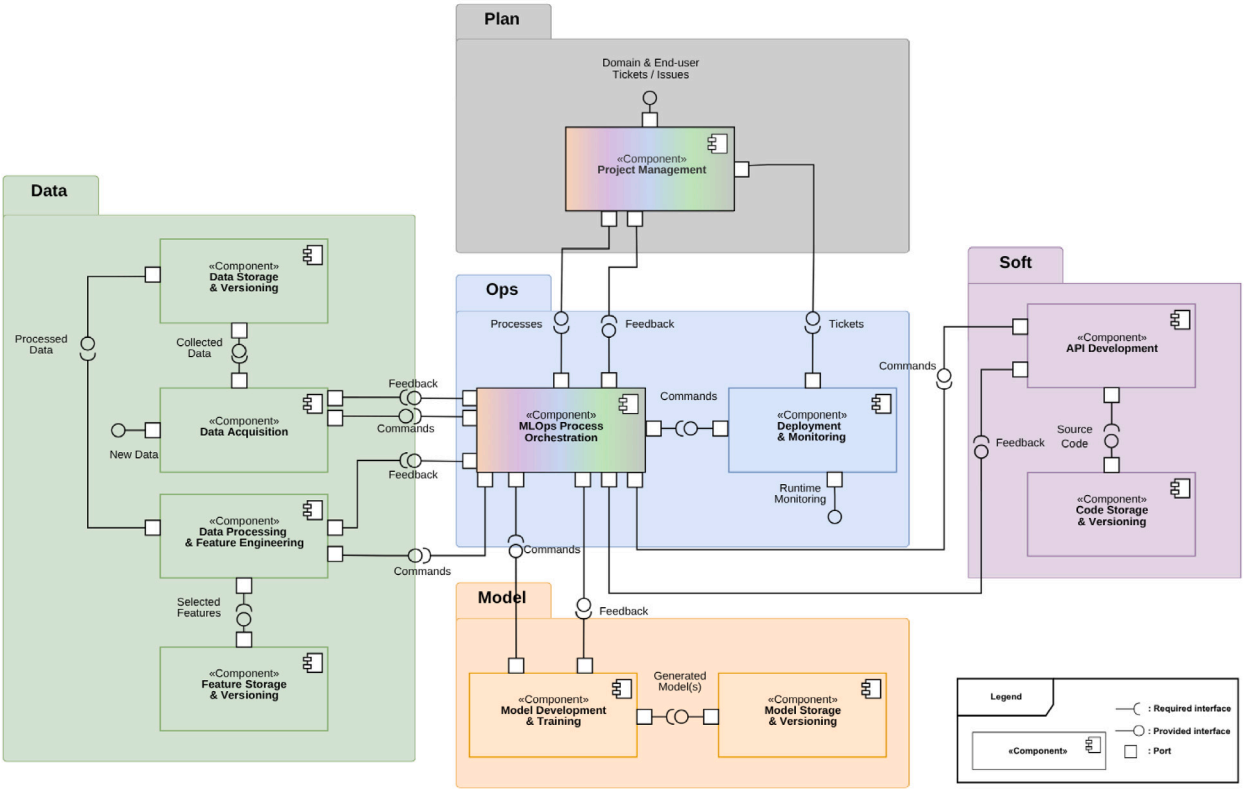


Fig. 6. Component-level architecture of SkeltyMLOps with an orchestration core coordinating Plan, Data, Model, Soft, and Ops packages.

5.7. A component-level view of SkeltyMLOps

Building on the package-level view, SkeltyMLOps refines each dimension into a package exposing interfaces for coordination. The objective is twofold: (i) preserve a separation of concerns aligned with the identified MLOps dimensions, and (ii) make cross-dimension collaboration explicit through clear boundaries, exchanged artifacts, and traceability metadata. The resulting component architecture is shown in Fig. 6; Table 9 maps the activities extracted in Section 4 to the components that operationalize them.

Artifact-producing modules

Around the orchestration core, SkeltyMLOps organizes the components that produce and evolve the core MLOps artifacts.

Data module. *Data Acquisition* ingests data from external sources; *Data Processing & Feature Engineering* transforms raw data into datasets and features; and *Data Storage & Versioning* together with *Feature Storage & Versioning* preserves these artifacts and their history. Versioning is a key concern in the architecture and should support both multi-granularity artifact histories [83] and controlled evolution across related artifacts [84].

Model module. *Model Development & Training* encapsulates training, validation, and optimization, while *Model Storage & Versioning* stores trained models and their versions together with associated evaluation evidence.

Software module. *API Development* implements and exposes model functionality, supported by *Code Storage & Versioning* for the evolution of the source code and packaging artifacts.

Operations module. *Deployment & Monitoring* manages release, deployment, runtime operation, monitoring, and incident handling. This module emits runtime events (*e.g.*, degradation events, drift indicators, anomalies) that may trigger contract-driven responses.

5.8. Core coordination model

Beyond functional decomposition, SkeltyMLOps defines how actors coordinate across modules. The coordination model is based on three concepts: tickets, events, and coordination contracts. A ticket is used to assign ownership and track progress. An event is emitted by a component to report a state change, such as a ready dataset or a detected drift. A coordination contract defines how a type of ticket progresses: required inputs, expected outputs, approvals, and escalation rules.

The *MLOps Process Orchestration (MPO)* executes these contracts. It dispatches work, manages handoffs between actors, and records execution evidence. It interacts with the *Project Management* component, which creates and prioritizes tickets, captures requirements and constraints, and assigns ownership.

In SkeltyMLOps, ticket flows are defined by explicit coordination contracts. For each type of ticket, MPO creates an execution instance under a specific contract version and enforces its rules. These rules may include automated tasks and human gates. A ticket cannot move forward until the required evidence and approvals are recorded. This allows projects to vary their processes without changing the architecture itself. Similar tickets follow the same flow under the same contract, and the executed path can later be reconstructed.

At runtime, MPO links the components' outputs to the ticket instances that triggered their production. In SkeltyMLOps, each handoff between actors is an explicit contract transition. A handoff specifies the producer actor, the consumer actor, the triggering condition, the artifact exchanged, and the metadata required for traceability. During operations, the same coordination rules continue to govern events (monitoring events are routed to the MPO).

Through the integration of the Project Management and MPO components, coordination is treated as a key concern of the architecture, making cross-actor handoffs explicit and recording execution evidence for traceability and governance.

5.9. Illustrative ticket lifecycle

To illustrate how these concepts operate together, consider a scenario in which a Product Owner has previously designed a drift-response process. Fig. 7 shows the corresponding sequence diagram, where every message passes through the MPO, concretely showing the coordination centered on orchestration. The following walkthrough traces a single ticket through an instance of that process.

The process begins when the Deployment & Monitoring component in the Ops module detects a data-drift event and raises a ticket to the Plan module, where the Product Owner reviews the drift alert, approves retraining, and assigns priority. The Project Management component then hands the ticket off to the MPO, which instantiates the corresponding drift-response process and performs a handoff to the Data module, where the Data Engineer updates the training dataset and registers it as dataset-v3 in Data Storage & Versioning. With the dataset artifact recorded, the MPO performs the handoff to the Model module, where the Model Engineer retrains the model using `dataset-v3` and registers `model-v2.1` together with its evaluation evidence in Model Storage & Versioning. The MPO then hands the ticket off to the Soft module, where the Software Engineer updates the serving endpoint and commits the changes to Code Storage & Versioning. Finally, the MPO performs the last handoff back to the Ops module, which deploys the updated software and resumes monitoring; the MPO closes the ticket with the full execution trace recorded. At each of these handoffs, the coordination contract specifies that the required artifact and approval must be present before the ticket advances to the next actor. The full structured trace (JSON), derived from this scenario, is available in the replication package.

This walkthrough illustrates how coordination benefits from the contract-driven design. Each cross-actor handoff is governed by the contract: the Data-to-Model transition, for instance, cannot proceed until the required dataset artifact and approval are recorded. In architectures without explicit coordination contracts, such handoffs are typically implicit and rely on informal signals between actors [85]. The contract-driven design makes ownership, required artifacts, and approval gates traceable at each transition.

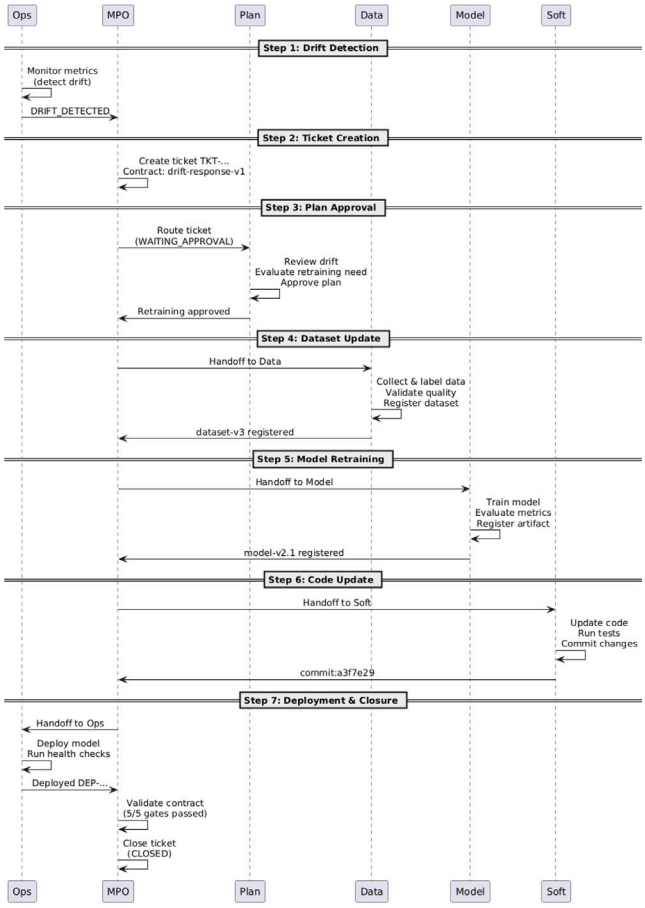


Fig. 7. Sequence diagram of the data-drift ticket lifecycle. All interactions pass through the MPO, which mediates each handoff between modules.

Algorithm 1 Conditional gate in the drift-response contract.

```
GATE 4 - Conditional: API Impact
if model.input_schema ≠ api.current_schema then
  GATE 4a - Code Update
  actor: Software Engineer
else
  skip to GATE 5
end if
```

When designing a contract, the Product Owner can include conditional gates that route the ticket along different paths depending on runtime conditions. For example, a retrained model may or may not change the expected input schema. If the schema changes, the serving API must be updated before deployment; otherwise, the code update step can be skipped. Algorithm 1 shows how the Product Owner would express this conditional gate in the drift-response contract. The walkthrough in Fig. 7 follows the branch where the schema has changed.

Three residual gaps remain at this stage: the architecture has not been instantiated in a production setting, contract design remains manual rather than generated from organizational policies, and interactions between concurrent tickets are not yet addressed.

5.10. Non-functional concerns

Although SkeltyMLOps is a reference architecture and does not prescribe a specific technology stack, the MPO design is compatible with architectural patterns (event-driven messaging, adapter-based interoperability, persistent state recovery) that address non-functional concerns relevant to practical deployment. The event-driven interactions between modules and the MPO are compatible with message-broker architectures, where contract execution can scale horizontally provided that tickets' states are externalized to a persistent store. Because ticket states are persistent, a failed transition leaves any ticket at its last recorded state, supporting fault recovery through retry. The standardized interfaces through which the MPO interacts with modules, namely ticket assignment, event consumption, and artifact registration, allow each module to be implemented by different tools via well-specified interfaces, mappings, and adapters, supporting interoperability with heterogeneous MLOps tool stacks without modifying the coordination logic. These features/characteristics/design choices make the coordination model compatible with existing processes and event-driven middleware, without tying it to specific tools.

The coordination model also supports governance concerns raised by regulatory and organizational requirements. Each coordination contract specifies which actor is responsible for each transition. The MPO is designed so that a ticket can advance only when the assigned actor provides the required evidence and approval. The MPO records execution evidence at every contract transition, including the acting actor, the timestamp, and the artifact produced, forming an append-only audit trail. Although enforcing strong security is beyond the scope of the current version, integrating security into the delivery process is an established concern in DevOps practice [86], and the append-only structure is compatible with mechanisms such as blockchain certification, an established approach for ensuring AI trustworthiness [87], should a specific implementation of SkeltyMLOps require them.

The combination of artifact versioning components with ticket-level execution traces creates a provenance chain from data source to deployed model, consistent with end-to-end ML provenance practices identified in the literature [88]. Tickets do not carry data: they carry only artifact references, and data remains in the storage components, avoiding direct data exposure within the orchestration flow. The domain-specific tools connected through the architecture's adapter interfaces already provide their own authentication, encryption, and access control mechanisms; the coordination layer delegates data-level and network security to these tools rather than reimplementing it.

> Overall, the architecture organizes MLOps as a set of modular components linked through explicit tickets, events, and coordination contracts, so that the activities identified in RQ3 are integrated into one coherent structure and the actors identified in RQ1 can collaborate through clear and traceable handoffs.

5.11. Threats to validity

Construct validity. Concerns not reported in the selected papers, such as regulatory compliance, fairness enforcement, or edge and federated deployment, fall outside the current scope. The coverage evaluation measures explicitness against a scope derived from the literature, not against industrial practice, so high coverage does not imply completeness across all MLOps settings. To mitigate this risk, the ongoing implementation orchestrates state-of-the-practice tools (such as MLFlow and DVC), which supports confidence in industrial settings.

External validity. SkeltyMLOps has not been instantiated or validated in a production MLOps setting. Its applicability to specialized contexts (*e.g.*, stream learning, federated learning) has not yet been assessed. The design, which does not prescribe specific tools, supports reuse but defers integration decisions to the instantiation level. To mitigate this risk, the implementation has been started with genericity and extensibility in mind.

6. Reproducibility and experimental artifacts

To support reuse and reproducibility, the replication package [33] provides prompts, evaluated model configurations, experiments and outputs. The package also contains the scripts used to call the evaluated LLMs and the implementation of the RAG.

7. Conclusion and future work

MLOps is increasingly practiced as a multi-actor engineering process in which delivery depends on coordinated work between specialized actors and tightly coupled artifacts (data, models, software, and operational insights). Although many tools exist, architectural guidance remains fragmented, and coordination breakdowns often occur at responsibility boundaries rather than within a single technical component. This paper addresses this gap through an SLR-based characterization of MLOps and a reference architecture that treats collaboration as a primary concern. The SLR is conducted to extract actors and process activities from the retained studies, followed by an analysis that combines human and LLM expertise to normalize terminology, derive five MLOps dimensions, and consolidate a large activity set into a compact list.

Based on these results, SkeltyMLOps is proposed as a modular reference architecture that aligns the modules with the derived dimensions and refines them into actor-aligned components that operationalize the consolidated activities. The architecture introduces orchestration as an explicit coordination backbone, centered on an MLOps Process Orchestrator (MPO) component paired with a Project Management component. A distinguishing aspect is that the progression of a ticket is governed by explicit versioned process definitions (*e.g.*, BPMN processes) uploaded as input artifacts. The MPO instantiates and executes these processes, enforces human gates, correlates artifacts and signals across dimensions, and supports auditability by reconstructing executed coordination paths. This design aims to preserve a stable architectural backbone while allowing project-to-project variability through process variants.

Finally, a coverage-based evaluation is reported that compares SkeltyMLOps with a set of MLOps architectures from the literature, assessing the extent to which the proposed decomposition accounts for the actors, dimensions, and activities reported. The evaluation supports the claim that an actor-aligned, orchestration-centered decomposition can serve as reusable architectural guidance for collaborative MLOps settings while remaining independent of a specific tool stack.

The ongoing work extends the architecture from a reference architecture to a more process-centric foundation for collaborative MLOps. This includes (i) a reusable palette of process templates that product owners can select and configure according to project needs, with execution delegated to the orchestrator so that coordination logic remains explicit; (ii) support for continuous learning settings, including incremental learning, by extending the orchestration model with processes for progressive updates and rollback of continuously evolving models; (iii) the evolution of the orchestrator into a standalone application that interoperates with actor-specific software, including the validation of scalability and reliability concerns in a production-like setting; (iv) the introduction of quality assurance for ML-powered software, including checks on model fairness; and (v) the integration of security concerns into the coordination layer.

CRediT authorship contribution statement

Charbel Daoud: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Danielle Azar:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Christelle Urtado: Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Sylvain Vauttier:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Acknowledgments

This research was partially supported by Région Occitanie (PLANETS project) and by the French National Research Agency (AdaptiveMLOps, ANR-24-IAS2-0004).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The replication package (prompts, LLM configurations, experiments, and outputs) is openly available on Zenodo [33].

References

[1] C. Daoud, D. Azar, J. Boiché, C. Urtado, S. Vauttier, SkeltyMLOps: Orchestrating collaborative mlops activities, in: 1st ECAI Workshop on Machine Learning Operations (MLOps25), in: CEUR Workshop Proceedings, vol. 4109, Bologna, Italy, 2025, pp. 55–66, URL https://ceur-ws.org/Vol-4109/paper6.pdf.

[2] P.C. Clements, D. Garlan, R. Little, R.L. Nord, J.A. Stafford, Documenting software architectures: Views and beyond, in: 25th International Conference on Software Engineering, ICSE, IEEE, Portland, USA, 2003, pp. 740–741, http://dx.doi.org/10.1109/ICSE.2003.1201264.

[3] R. Cloutier, G. Muller, D. Verma, R. Nilchiani, E. Hole, M. Bone, The concept of reference architectures, Syst. Eng. 13 (1) (2010) 14–27, http://dx.doi.org/10.1002/sys.20129.

[4] E.Y. Nakagawa, Reference architectures and variability: Current status and future perspectives, in: T. Männistö, M.A. Babar, C.E. Cuesta, J.E. Savolainen (Eds.), WICSA/ECSA Companion Volume, in: ACM International Conference Proceeding Series, (704) ACM, Helsinki, Finland, 2012, pp. 159–162, http://dx.doi.org/10.1145/2361999.2362032.

[5] S. Amershi, A. Begel, C. Bird, R. DeLine, H.C. Gall, E. Kamar, N. Nagappan, B. Nushi, T. Zimmermann, Software engineering for machine learning: A case study, in: IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, Montreal, Canada, 2019, pp. 291–300, http://dx.doi.org/10.1109/ICSE-SEIP.2019.00042.

[6] L.S. Myllyaho, M. Raatikainen, T. Männistö, J.K. Nurminen, T. Mikkonen, On misbehaviour and fault tolerance in machine learning systems, J. Syst. Softw. 183 (2022) 111096, http://dx.doi.org/10.1016/j.jss.2021.111096.

[7] A. Serban, K. van der Blom, H.H. Hoos, J. Visser, Adoption and effects of software engineering best practices in machine learning, in: 14th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM, ACM, Bari, Italy, 2020, pp. 3:1–3:12, http://dx.doi.org/10.1145/3382494.3410681.

[8] A. Basile, F. Calefato, F. Lanubile, G. Mallardi, L. Quaranta, An MLOps solution framework for transitioning machine learning models into ehealth systems, in: Ital-IA 2024 Thematic Workshops, in: CEUR Workshop Proceedings, 3762, CEUR-WS.org, Naples, Italy, 2024, pp. 318–323, URL https://ceur-ws.org/Vol-3762/524.pdf.

[9] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, D. Dennison, Hidden technical debt in machine learning systems, in: Advances in Neural Information Processing Systems (NeurIPS), Curran Associates, Inc., Montreal, Canada, 2015, pp. 2503–2511, URL https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf.

[10] D. O'Brien, S. Biswas, S.M. Imtiaz, R. Abdalkareem, E. Shihab, H. Rajan, 23 shades of self-admitted technical debt: An empirical study on machine learning software, in: 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, Singapore, Singapore, 2022, pp. 734–746, http://dx.doi.org/10.1145/3540250.3549088.

[11] S. Li, J. Guo, J.-G. Lou, M. Fan, T. Liu, D. Zhang, Testing machine learning systems in industry: An empirical study, in: 44th ACM/IEEE International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, Pittsburgh, PA, USA, 2022, pp. 263–272, http://dx.doi.org/10.1109/ICSE-SEIP55303.2022.9793981.

[12] B. Eken, S. Pallewatta, N.K. Tran, A. Tosun, M.A. Babar, A multivocal review of MLOps practices, challenges and open issues, ACM Comput. Surv. 58 (2) (2025) 39:1–39:35, http://dx.doi.org/10.1145/3747346.

[13] D. Kreuzberger, N. Kühl, S. Hirschl, Machine learning operations (MLOps): Overview, definition, and architecture, IEEE Access 11 (2023) 31866–31879, http://dx.doi.org/10.1109/ACCESS.2023.3262138.

[14] M. Steidl, M. Felderer, R. Ramler, The pipeline for the continuous development of artificial intelligence models—Current state of research and practice, J. Syst. Softw. 199 (2023) 111615, http://dx.doi.org/10.1016/j.jss.2023.111615.

[15] A.M. Burgueño-Romero, A. Benítez-Hidalgo, C. Barba-González, J.F. Aldana-Montes, Toward an open source MLOps architecture, IEEE Softw. 42 (1) (2025) 59–64, http://dx.doi.org/10.1109/MS.2024.3421675.

[16] A. Robustelli, A. Marfoglia, C. D'Errico, S. Mellone, A. Carbonaro, Feasibility of MLOps-based healthcare pipelines in ensuring the cybersecurity framework, in: 1st ECAI Workshop on Machine Learning Operations (MLOps25@ECAI 2025), in: CEUR Workshop Proceedings, vol. 4109, CEUR-WS.org, Bologna, Italy, 2025, pp. 1–12, URL https://ceur-ws.org/Vol-4109/paper1.pdf.

[17] L.E. Lwakatare, I. Crnkovic, J. Bosch, DevOps for AI – challenges in development of AI-enabled applications, in: 28th International Conference on Software, Telecommunications and Computer Networks (SoftCOM), IEEE, Hvar, Croatia, 2020, pp. 1–6, http://dx.doi.org/10.23919/SoftCOM50211.2020.9238323.

[18] A. Paleyes, R.-G. Urma, N.D. Lawrence, Challenges in deploying machine learning: A survey of case studies, ACM Comput. Surv. 55 (6) (2022) 114:1–114:29, http://dx.doi.org/10.1145/3533378.

[19] M. Hüttermann, DevOps for Developers, A Press, USA, 2012, http://dx.doi.org/10.1007/978-1-4302-4570-4.

[20] L. Zhu, L. Bass, G. Champlin-Scharff, DevOps and its practices, IEEE Softw. 33 (3) (2016) 32–34, http://dx.doi.org/10.1109/MS.2016.81.

[21] B.S. Farroha, D.L. Farroha, A framework for managing mission needs, compliance, and trust in the DevOps environment, in: IEEE Military Communications Conference (MILCOM), IEEE, Baltimore, USA, 2014, pp. 288–293, http://dx.doi.org/10.1109/MILCOM.2014.54.

[22] N. Azad, S. Hyrynsalmi, DevOps critical success factors—A systematic literature review, Inf. Softw. Technol. 157 (2023) 107150, http://dx.doi.org/10.1016/j.infsof.2023.107150.

[23] C. Ebert, G. Gallardo, J. Hernantes, N. Serrano, DevOps, IEEE Softw. 33 (3) (2016) 94–100, http://dx.doi.org/10.1109/MS.2016.68.

[24] E. Breck, S. Cai, E. Nielsen, M. Salib, D. Sculley, The ML test score: A rubric for ML production readiness and technical debt reduction, in: IEEE International Conference on Big Data, IEEE, Boston, USA, 2017, pp. 1123–1132, http://dx.doi.org/10.1109/BigData.2017.8258038.

[25] A.K. Narayanappa, C. Amrit, An analysis of the barriers preventing the implementation of MLOps, in: S.K. Sharma, Y.K. Dwivedi, B. Metri, B. Lal, A. Elbanna (Eds.), Transfer, Diffusion and Adoption of Next-Generation Digital Technologies, in: IFIP Advances in Information and Communication Technology, vol. 697, Springer, Nagpur, India, 2024, pp. 101–114, http://dx.doi.org/10.1007/978-3-031-50188-3_10.

[26] A. Melde, M. Madan, P. Gavrikov, D. Hoof, A. Laubenheimer, J. Keuper, C. Reich, Tackling key challenges of AI development—Insights from an industry–academia collaboration, in: C. Reich, U. Mescheder (Eds.), The Upper-Rhine Artificial Intelligence Symposium (UR-AI 2022): AI Applications in Medicine and Manufacturing, Furtwangen University, Villingen-Schwenningen, Germany, 2022, pp. 112–121, URL https://opus.hs-furtwangen.de/files/8624/Tackling.pdf.

[27] F. Calefato, F. Lanubile, L. Quaranta, A preliminary investigation of MLOps practices in GitHub, in: 16th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM, ACM, Helsinki, Finland, 2022, pp. 283–288, http://dx.doi.org/10.1145/3544902.3546636.

[28] F.A. Najafabadi, J. Bogner, I. Gerostathopoulos, P. Lago, An analysis of MLOps architectures: A systematic mapping study, in: M. Galster, P. Scandurra, T. Mikkonen, P.O. Antonino, E.Y. Nakagawa, E. Navarro (Eds.), Software Architecture: 18th European Conference, ECSA, in: Lecture Notes in Computer Science, vol. 14889, Springer, Luxembourg City, Luxembourg, 2024, pp. 69–85, http://dx.doi.org/10.1007/978-3-031-70797-1_5.

[29] A.P. Woźniak, M. Milczarek, J. Woźniak, MLOps components, tools, process and metrics—A systematic literature review, IEEE Access 13 (2025) 22166–22175, http://dx.doi.org/10.1109/ACCESS.2025.3534990.

[30] K. Petersen, R. Feldt, S. Mujtaba, M. Mattsson, Systematic mapping studies in Software Engineering, in: 12th International Conference on Evaluation and Assessment in Software Engineering (EASE), BCS Learning and Development Ltd., Bari, Italy, 2008, pp. 68–77, http://dx.doi.org/10.14236/ewic/EASE2008.8.

[31] A. Ampatzoglou, S. Bibi, P. Avgeriou, M. Verbeek, A. Chatzigeorgiou, Identifying, categorizing and mitigating threats to validity in Software Engineering secondary studies, Inf. Softw. Technol. 106 (2019) 201–230, http://dx.doi.org/10.1016/j.infsof.2018.10.006.

[32] B.A. Kitchenham, S. Charters, Guidelines for Performing Systematic Literature Reviews in Software Engineering, Version 2.3, Tech. Rep. EBSE-2007-01, Software Engineering Group, School of Computer Science and Mathematics, Keele University, 2007, URL https://legacyfileshare.elsevier.com/promis_misc/525444systematicreviewsguide.pdf.

[33] C. Daoud, D. Azar, C. Urtado, S. Vauttier, A reference architecture for an orchestrated collaborative MLOps: A ground-up design from literature combining human and LLM expertise, 2026, http://dx.doi.org/10.5281/zenodo.21311713, [Dataset], Future Generation Computer Systems, Zenodo, Version 1.1.

[34] A. Martín-Martín, E. Orduña-Malea, M. Thelwall, E.D. López-Cózar, Google Scholar, Web of Science, and Scopus: A systematic comparison of citations in 252 subject categories, J. Inf. 12 (4) (2018) 1160–1177, http://dx.doi.org/10.1016/j.joi.2018.09.002.

[35] J.-F. Gehanno, L. Rollin, S. Darmoni, Is the coverage of Google Scholar enough to be used alone for systematic reviews, BMC Med. Inform. Decis. Mak. 13 (1) (2013) 7, http://dx.doi.org/10.1186/1472-6947-13-7.

[36] G. Recupito, F. Pecorelli, G. Catolino, S. Moreschini, D. Di Nucci, F. Palomba, D.A. Tamburri, A multivocal literature review of MLOps tools and features, in: 48th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, IEEE, Maspalomas, Spain, 2022, pp. 84–91, http://dx.doi.org/10.1109/SEAA56994.2022.00021.

[37] D.O. Hanchuk, S.O. Semerikov, Implementing MLOps practices for effective machine learning model deployment: A meta-synthesis, in: 7th International Workshop on Augmented Reality in Education (AREdu), in: CEUR Workshop Proceedings, vol. 3918, CEUR-WS.org, 2025, pp. 329–337, URL https://ceur-ws.org/Vol-3918/paper404.pdf.

[38] X. Zhang, J. Jaskolka, Conceptualizing the secure machine learning operations (SecMLOps) paradigm, in: IEEE 22nd International Conference on Software Quality, Reliability and Security, QRS, IEEE, 2022, pp. 127–138, http://dx.doi.org/10.1109/QRS57517.2022.00023.

[39] M. Antonini, M. Pincheira, M. Vecchio, F. Antonelli, Tiny-MLOps: A framework for orchestrating ML applications at the far edge of IoT systems, in: IEEE International Conference on Evolving and Adaptive Intelligent Systems, EAIS, IEEE, Larnaca, Cyprus, 2022, pp. 1–8, http://dx.doi.org/10.1109/EAIS51927.2022.9787703.

[40] A. Lima, L. Monteiro, A.P. Furtado, MLOps: Practices, maturity models, roles, tools, and challenges – a systematic literature review, in: 24th International Conference on Enterprise Information Systems - Volume 1: ICEI, SciTePress, 2022, pp. 308–320, http://dx.doi.org/10.5220/0010997300003179.

[41] P. Ruf, M. Madan, C. Reich, D. Ould-Abdeslam, Demystifying MLOps and presenting a recipe for the selection of open-source tools, Appl. Sci. 11 (19) (2021) 8861, http://dx.doi.org/10.3390/app11198861.

[42] M. Slupczynski, R. Klamma, MILKI-PSY cloud: Mlops-based multimodal sensor stream processing pipeline for learning analytics in psychomotor education, in: Multimodal Immersive Learning Systems Workshop 2022 (MILeS@EC-TEL), in: CEUR Workshop Proceedings, vol. 3247, CEUR-WS.org, Toulouse, France, 2022, pp. 8–14, URL https://ceur-ws.org/Vol-3247/paper1.pdf.

[43] Y. Akkem, S.K. Biswas, A. Varanasi, Smart farming monitoring using ML and MLOps, in: Intelligent Computing and Communication, in: Lecture Notes in Networks and Systems, vol. 703, Springer Nature Singapore, 2023, pp. 665–675, http://dx.doi.org/10.1007/978-981-99-3315-0_51.

[44] M. Barry, J. Montiel, A. Bifet, S. Wadkar, N. Manchev, M. Halford, R. Chiky, S.E. Jaouhari, K.B. Shakman, J.A. Fehaily, F.L. Deit, V.-T. Tran, E. Guerizec, StreamMLOps: Operationalizing online learning for big data streaming and real-time applications, in: IEEE 39th International Conference on Data Engineering, ICDE, IEEE, 2023, pp. 3508–3521, http://dx.doi.org/10.1109/ICDE55515.2023.00272.

[45] M.N. Chowdary, B. Sankeerth, C.K. Chowdary, M. Gupta, Accelerating the machine learning model deployment using MLOps, J. Phys.: Conf. Ser. 2327 (1) (2022) 012027, http://dx.doi.org/10.1088/1742-6596/2327/1/012027.

[46] E. Yamaguchi, Speeding up the machine-learning process with MLOps and creating a mechanism to continuously provide service value, NTT Tech. Rev. 19 (5) (2021) 61–67, http://dx.doi.org/10.53829/ntr202105fa10, URL https://www.ntt-review.jp/archive/ntttechnical.php?contents=ntr202105fa10.html.

[47] A. Grilo, P. Figueiras, B. Rêga, L. Lourenço, A. Khodamoradi, R. Costa, R. Jardim-Gonçalves, Data analytics environment: Combining visual programming and MLOps for AI workflow creation, in: IEEE International Conference on Engineering, Technology, and Innovation (ICE/ITMC), IEEE, 2024, pp. 1–9, http://dx.doi.org/10.1109/ICE/ITMC61926.2024.10794244.

[48] S. Jain, J. Das, Integrating data engineering and MLOps for scalable and resilient machine learning pipelines: frameworks, challenges, and future trends, World J. Adv. Eng. Technol. Sci. 14 (1) (2025) 241–253, http://dx.doi.org/10.30574/wjaets.2025.14.1.0020.

[49] M.M. John, H.H. Olsson, J. Bosch, Towards MLOps: A framework and maturity model, in: 47th Euromicro Conference on Software Engineering and Advanced Applications, SEAA, IEEE, Palermo, Italy, 2021, pp. 1–8, http://dx.doi.org/10.1109/SEAA53835.2021.00050.

[50] I. Karamitsos, S. Albarhami, C. Apostolopoulos, Applying DevOps practices of continuous automation for machine learning, Information 11 (7) (2020) 363, http://dx.doi.org/10.3390/info11070363.

[51] M.Y.S. Krishna, S.K. Gawre, MLOps for enhancing the accuracy of machine learning models using DevOps, continuous integration, and continuous deployment, Res. Rep. Comput. Sci. 2 (3) (2023) 97–103, http://dx.doi.org/10.37256/rrcs.2320232644.

[52] S. Kwatra, U. Mangla, M. Srivatsa, Model operationalization at edge devices, in: Artificial Intelligence for Edge Computing, Springer Nature Switzerland, 2023, pp. 353–365, http://dx.doi.org/10.1007/978-3-031-40787-1_12.

[53] P. Liang, B. Song, X. Zhan, Z. Chen, J. Yuan, Automating the training and deployment of models in MLOps by integrating systems with machine learning, Applied and Computational Engineering 76 (2024) 1–7, http://dx.doi.org/10.54254/2755-2721/76/20240690.

[54] Y. Liu, Z. Ling, B. Huo, B. Wang, T. Chen, E. Mouine, Building a platform for machine learning operations from open source frameworks, IFAC-PapersOnLine 53 (5) (2020) 704–709, http://dx.doi.org/10.1016/j.ifacol.2021.04.161.

[55] S. Mei, C. Liu, Q. Wang, H. Su, Model provenance management in MLOps pipeline, in: 2022 8th International Conference on Computing and Data Engineering, ICCDE, ACM, 2022, pp. 45–50, http://dx.doi.org/10.1145/3512850.3512861.

[56] Ó.A. Méndez, J. Camargo, H. Florez, Machine learning operations applied to development and model provisioning, in: Applied Informatics: 7th International Conference, in: Communications in Computer and Information Science, vol. 2236, Springer Nature Switzerland, 2024, pp. 73–88, http://dx.doi.org/10.1007/978-3-031-75144-8_6.

[57] R. Miñón, J.D. de Arcaya, A.I. Torre-Bastida, P. Hartlieb, Pangea: An MLOps tool for automatically generating infrastructure and deploying analytic pipelines in edge, fog and cloud layers, Sensors 22 (12) (2022) 4425, http://dx.doi.org/10.3390/s22124425.

[58] D.R. Niranjan, Mohana, Jenkins pipelines: A novel approach to machine learning operations (MLOps), in: International Conference on Edge Computing and Applications, ICECAA, IEEE, 2022, pp. 1292–1297, http://dx.doi.org/10.1109/ICECAA55415.2022.9936252.

[59] I. Pölöskei, MLOps approach in the cloud-native data pipeline design, Acta Tech. Jaurinensis 15 (1) (2022) 1–6, http://dx.doi.org/10.14513/actatechjaur.00581.

[60] T. Raffin, T. Reichenstein, J. Werner, A. Kühl, J. Franke, A reference architecture for the operationalization of machine learning models in manufacturing, Procedia CIRP 115 (2022) 130–135, http://dx.doi.org/10.1016/j.procir.2022.10.062.

[61] M. Safdar, P.P. Paul, G. Lamouche, G. Wood, M. Zimmermann, F. Hannesen, C. Bescond, P. Wanjara, Y.F. Zhao, Fundamental requirements of a machine learning operations platform for industrial metal additive manufacturing, Comput. Ind. 154 (2024) 104037, http://dx.doi.org/10.1016/j.compind.2023.104037.

[62] R. Semola, V. Lomonaco, D. Bacciu, Continual-learning-as-a-service (CLaaS): On-demand efficient adaptation of predictive models, in: Joint Proceedings of CI4PM 2022 and PAI 2022, co-located with WCCI 2022, in: CEUR Workshop Proceedings, 3350, CEUR-WS.org, 2022, pp. 46–55, URL https://ceur-ws.org/Vol-3350/PAI1.pdf.

[63] R. Subramanya, S. Sierla, V. Vyatkin, From DevOps to MLOps: Overview and application to electricity market forecasting, Appl. Sci. 12 (19) (2022) 9851, http://dx.doi.org/10.3390/app12199851.

[64] A.H. van Bruggen, K. Kruger, A.H. Basson, J. Grobler, An architecture to integrate digital twins and machine learning operations, in: Service Oriented, Holonic and Multi-Agent Manufacturing Systems for Industry of the Future, in: Studies in Computational Intelligence, 1136, Springer, Switzerland, 2024, pp. 3–14, http://dx.doi.org/10.1007/978-3-031-53445-4_1.

[65] G. Zárate, R. Miñón, J.D. de Arcaya, A.I. Torre-Bastida, K2E: Building MLOps environments for governing data and models catalogues while tracking versions, in: IEEE 19th International Conference on Software Architecture Companion (ICSA-C), IEEE, 2022, pp. 206–209, http://dx.doi.org/10.1109/ICSA-C54293.2022.00047.

[66] Y. Zhou, Y. Yu, B. Ding, Towards MLOps: A case study of ML pipeline platform, in: International Conference on Artificial Intelligence and Computer Engineering, ICAICE, IEEE, Beijing, China, 2020, pp. 494–500, http://dx.doi.org/10.1109/ICAICE51518.2020.00102.

[67] S. Moreschini, D. Hästbacka, D. Taibi, MLOps pipeline development: The OSSARA use case, in: 2023 International Conference on Research in Adaptive and Convergent Systems, RACS, ACM, Gdansk, Poland, 2023, http://dx.doi.org/10.1145/3599957.3606211.

[68] R. Dautov, E.J. Husom, F. Gonidis, Towards MLOps in mobile development with a plug-in architecture for data analytics, in: 2022 6th International Conference on Computer, Software and Modeling, ICCSM, IEEE, 2022, pp. 22–27, http://dx.doi.org/10.1109/ICCSM57214.2022.00011.

[69] S. Moreschini, F. Lomio, D. Hästbacka, D. Taibi, MLOps for evolvable AI-intensive software systems, in: IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER, IEEE, Honolulu, USA, 2022, pp. 1293–1294, http://dx.doi.org/10.1109/SANER53432.2022.00155.

[70] A.Y. Sitcheu, N. Friederich, S. Baeuerle, O. Neumann, M. Reischl, R. Mikut, MLOps for scarce image data: A use case in microscopic image analysis, in: 33rd Workshop Computational Intelligence (CI 2023), KIT Scientific Publishing, 2023, pp. 169–190, http://dx.doi.org/10.5445/IR/1000164691.

[71] K.R. Felizardo, E. Mendes, M. Kalinowski, É.F. de Souza, N.L. Vijaykumar, Using forward snowballing to update systematic reviews in software engineering, in: 10th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM, ACM, Ciudad Real, Spain, 2016, pp. 53:1–53:6, http://dx.doi.org/10.1145/2961111.2962630.

[72] C. Wohlin, M. Kalinowski, K.R. Felizardo, E. Mendes, Successful combination of database search and snowballing for identification of primary studies in systematic literature studies, Inf. Softw. Technol. 147 (2022) 106908, http://dx.doi.org/10.1016/j.infsof.2022.106908.

[73] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: 18th International Conference on Evaluation and Assessment in Software Engineering, EASE, ACM, London, United Kingdom, 2014, pp. 1–10, http://dx.doi.org/10.1145/2601248.2601268.

[74] H.-F. Hsieh, S.E. Shannon, Three approaches to qualitative content analysis, Qual. Health Res. 15 (9) (2005) 1277–1288, http://dx.doi.org/10.1177/1049732305276687.

[75] J.R. Landis, G.G. Koch, The measurement of observer agreement for categorical data, Biometrics 33 (1) (1977) 159–174, http://dx.doi.org/10.2307/2529310.

[76] Y. Zhang, Z. Wang, J. Shang, ClusterLLM: Large language models as a guide for text clustering, in: Empirical Methods in Natural Language Processing (EMNLP), Association for Computational Linguistics, Singapore, 2023, pp. 13903–13920, http://dx.doi.org/10.18653/v1/2023.emnlp-main.858.

[77] V. Viswanathan, K. Gashteovski, C. Lawrence, T. Wu, G. Neubig, Large language models enable few-shot clustering, Trans. Assoc. Comput. Linguist. 12 (2024) 321–333, http://dx.doi.org/10.1162/tacl_a_00648.

[78] J.L. Campbell, C. Quincy, J. Osserman, O.K. Pedersen, Coding in-depth semistructured interviews: Problems of unitization and intercoder reliability and agreement, Sociol. Methods Res. 42 (3) (2013) 294–320, http://dx.doi.org/10.1177/0049124113500475.

[79] G. Marvin, N. Hellen, D. Jjingo, J. Nakatumba-Nabende, Prompt engineering in large language models, in: I.J. Jacob, S. Piramuthu, P. Falkowski-Gilski (Eds.), International Conference on Data Intelligence and Cognitive Informatics ICDICI, in: Algorithms for Intelligent Systems, Springer Singapore, Tirunelveli, India, 2024, pp. 387–402, http://dx.doi.org/10.1007/978-981-99-7962-2_30.

[80] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, D.C. Schmidt, A prompt pattern catalog to enhance prompt engineering with ChatGPT, 2023, http://dx.doi.org/10.48550/arXiv.2302.11382, ArXiv Preprint, arXiv:2302.11382.

[81] M.G. Rodrigues, E.K. Viegas, A.O. Santin, F. Enembreck, A MLOps architecture for near real-time distributed stream learning operation deployment, J. Netw. Comput. Appl. 238 (2025) 104169, http://dx.doi.org/10.1016/j.jnca.2025.104169.

[82] I. Kumara, R. Arts, R.C. Ferreira, D.D. Nucci, R. Kazman, D.A. Tamburri, W.-J. van den Heuvel, MLOps in practice: Requirements and a reference architecture from industry, in: Software Architecture: 19th European Conference, ECSA, in: Lecture Notes in Computer Science, vol. 15929, Springer Nature Switzerland, Limassol, Cyprus, 2025, pp. 20–37, http://dx.doi.org/10.1007/978-3-032-02138-0_2.

[83] C. Urtado, C. Oussalah, Complex entity versioning at two granularity levels, Inf. Syst. 23 (3–4) (1998) 197–216, http://dx.doi.org/10.1016/S0306-4379(98)00009-X.

[84] A.L. Borgne, D. Delahaye, M. Huchard, C. Urtado, S. Vauttier, Substitutability-based version propagation to manage the evolution of three-level component-based architectures, in: 29th International Conference on Software Engineering and Knowledge Engineering, SEKE, Knowledge Systems Institute, Pittsburgh, USA, 2017, pp. 18–23, http://dx.doi.org/10.18293/SEKE2017-118.

[85] N. Nahar, S. Zhou, G. Lewis, C. Kästner, Collaboration challenges in building ML-enabled systems: Communication, documentation, engineering, and process, in: 44th International Conference on Software Engineering, ICSE, ACM, Pittsburgh, USA, 2022, pp. 413–425, http://dx.doi.org/10.1145/3510003.3510209.

[86] X. Zhao, T. Clear, R. Lal, Identifying the primary dimensions of DevSecOps: A multi-vocal literature review, J. Syst. Softw. 214 (2024) 112063, http://dx.doi.org/10.1016/j.jss.2024.112063.

[87] P. Zhang, S. Ding, Q. Zhao, Exploiting blockchain to make AI trustworthy: A software development lifecycle view, ACM Comput. Surv. 56 (7) (2024) 1–31, http://dx.doi.org/10.1145/3614424.

[88] M. Schlegel, K.-U. Sattler, Capturing end-to-end provenance for machine learning pipelines, Inf. Syst. 132 (2025) 102495, http://dx.doi.org/10.1016/j.is.2024.102495.